

Dominic Ayre

Senior Software Engineer | Microsoft Azure Container Instances | Cambridge
MEng Computer Science - University of Bristol

Microsoft - Azure Container Instances (ACI)

Senior Software Engineer (Jan 2026 - Present)

Technical lead for a team of 6 engineers and responsible for several engineering work-streams that my team works on. Azure Container Instances is a service for running Docker/OCI containers.

- I am responsible for building out Azure Container Instances, I have lead the buildout of 5 new regions, added zonal redundancy to 2 and expanded availability zones in 1. I have also automated the buildout process cutting out buildout time for new regions from 2 weeks to 2 days.
- I am also responsible for a feature called NGroups, which allows users to manage a set of container groups as a single resource. This feature is used by Azure's Quota Service requiring the highest standards for quality and availability.
- I have also worked individually at the node level to add support for new, more cost efficient node SKUs which enables significant cost saving.

Microsoft - Azure Research/Azure Confidential Computing

Research Software Engineer (Aug 2022 - Jan 2026)

Individual Contributor working on solutions for distributed services built on AMD SNP Confidential hardware.

- I added support for AMD SNP attestation reports to the Confidential Consortium Framework (CCF), an open source C++ framework used for building systems with decentralised trust. It is the basis for several Microsoft services such as Azure Confidential Ledger.
- I implemented a Key Management Service build on top of CCF for Microsoft's implementation of Google's Privacy Sandbox.
- I also worked on the library used to generate security policies from deployment templates, including a 10x performance improvement in image layer measuring.
- Built and maintained open source repositories for Confidential-ACI for testing, sidecars, examples.

Arm - GPU Software

Senior Software Engineer (Sep 2017 - Aug 2022)

At Arm, I built tools to instrument the GPU driver, parse and then visualise the data to enable kernel level optimisations.

- I implemented an extensible TypeScript based web UI for multiple data sources from the GPU driver which could be organised together to correlate information such as Vulkan command buffers, power information, hardware counters.
- I was also heavily involved in the Python library used to parse the raw data coming out of the GPU driver, and adding the instrumentation itself to our Kernel and Userspace drivers written in C and C++.